CS260 - Written Assignment 3

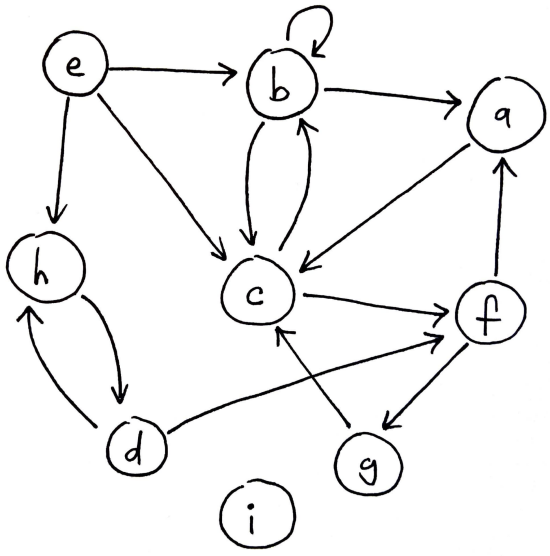
Professor Galen Long, Professsor Kurt Schmidt, Professor Yusuf Osmanlioglu

Summer 2024-25

Total points: 100

- **For any algorithm design questions in this assignment and the course as a whole, give your algorithms in C/Python-like pseudocode *and* describe your algorithm briefly at a high level in plain English.**
- Your pseudocode can omit low-level details like memory allocations and type declarations.
- If you're not sure whether or not to include a detail in your pseudocode, err on the side of including it.
- Small mistakes like off-by-one errors will receive at most a small penalty. We know it's hard to write perfectly correct pseudocode because you can't execute it easily (although running your pseudocode on an example on paper is a great way to debug it).
- Whenever we ask you to design an algorithm, we expect your algorithm to be efficient (within reason). If an asymptotically more efficient algorithm exists, you may lose points if your algorithm is correct but asymptotically slower.
- However, you should prefer a correct but slow algorithm over an incorrect attempt at a more efficient algorithm.

1. Consider the following graph, $G = (V, E)$:



   (a) (15 points) Run BFS on G from node `h`, using the pseudocode given in `bfs_dfs_pseudocode.tex` that's linked in the assignment description on Blackboard.

Give each node's d and p values in a table with the following format:

Node	d	p
a	-1	x
b	-1	x
c	-1	x
d	-1	x
e	-1	x
f	-1	x
g	-1	x
h	-1	x
i	-1	x

When the choice of which node to explore next is arbitrary, go in alphabetical order.

(b) (15 points) Run DFS on G, using the pseudocode given in `bfs_dfs_pseudocode.tex` that's linked in the assignment description on Blackboard.

Give each node's d, f, and p values in a table with the following format:

Node	d	f	p
a	-1	-1	x
b	-1	-1	x
c	-1	-1	x
d	-1	-1	x
e	-1	-1	x
f	-1	-1	x
g	-1	-1	x
h	-1	-1	x
i	-1	-1	x

When the choice of which node to explore next is arbitrary, go in alphabetical order.

(c) (5 points) Label each edge in G by whether it's a tree edge, forward/cross edge, or back edge, based on your DFS traversal.

(d) (5 points) Draw the DFS interval graph given by your DFS traversal.

2. (5 points) Given a digraph G, we can create its *transpose*, called G^T, by reversing all of G's edges. E.g. $(u, v) \in G.E$ if and only if $(v, u) \in G^T.E$.

   Give an example of a non-empty digraph G such that $G = G^T$, meaning that the transpose would have the exact same nodes and edges as the original graph.

3. Assume BFS outputted this predecessor table when run on some graph G:

Node	p
p	NIL
q	s
r	u
s	v
t	q
u	p
v	p

(a) (3 points) Give the shortest path from p to t and its length.

(b) (2 points) Does this predecessor table contain all edges in G? Why or why not?

4. The CS department is planning a curriculum redesign. Each CS course x has 0 or more CS courses as prerequisites, which must all be taken before x.

   You've been asked to design an algorithm that will check if, given each CS course and its prerequisites, there's any sequence of courses that would be impossible to take in any order.

(a) (5 points) Describe how you'd model this problem as a graph by answering all of the following questions:

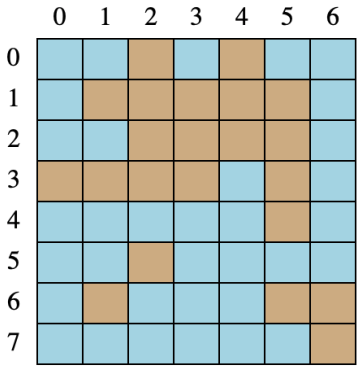
- What are the nodes?
- What does an edge between two nodes represent about those nodes?
- Is the graph directed or undirected?
- Is the graph weighted or unweighted? If weighted, what are the weights?

(b) (15 points) Design an algorithm that takes the graph you previously described as input and outputs false if the graph has an impossible preqrequisite sequence and true otherwise. Briefly explain why your algorithm solves the problem.

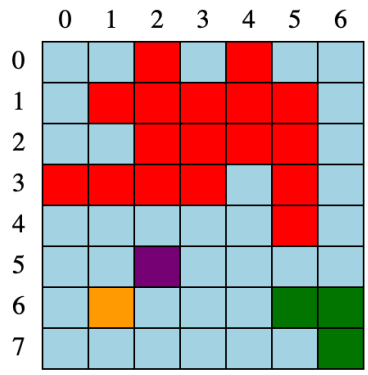
If your algorithm reuses other algorithms given in the class with some tweaks, reproduce their pseudocode in full with any changes you might've made.

(c) (5 points) Give tight bounds on the best and worst case runtimes of your algorithm and briefly justify your bounds.

5. Consider a 2D grid where each square represents either water or land. For example, in the below 8x7 grid, each tan square is a land square and each blue square is a water square (the numbers on the outside of the grid label the indices of each square):



If two land squares are vertically or horizontally (but not diagonally) adjacent, they are considered to be part of the same island. There are four islands in this grid (given different colors below):



The purple island on $(5, 2)$ and the orange island on $(6, 1)$ are considered separate because they're only diagonally adjacent, not vertically or horizontally adjacent.

(a) (15 points) Design an algorithm that will take an $n \times m$ array of booleans, where true means the square is an island and false means it's water, and a $(row, column)$ pair of indices. It should return a count of the number of squares in the island the $(row, column)$ pair is a part of, if any.

For example, when given the above grid and $(3, 1)$ as input, your algorithm should return 17. If given $(6, 1)$ as input, your algorithm should return 1. If given $(3, 4)$ as input, your algorithm should return 0.

If your algorithm reuses other algorithms given in the class with some tweaks, you **don't** have to reproduce their pseudocode in full – just describe in detail any changes you'd make.

Hint: How could you convert the inputted 2D array of booleans into a graph, whether an adjacency matrix or adjacency list? What would be your vertices and their labels? How would you represent edges? Note that each $(row, column)$ cell can be given a unique label of $row * n + column$.

(b) (10 points) Give tight bounds on the best and worst case running times of your algorithm in terms of n and m and briefly justify your bounds.